

PRÁCTICA 1: MATEMÁTICA DISCRETA

CURSO 2025-2026

**“IMAT-LAB: ARITMÉTICA
MODULAR”**

Autores:

GABRIELA ROMERO MARTÍN

CANDELA TEJEDO RAGA

GP10

Índice de Contenidos

1. INTRODUCCIÓN.....	3
2. INTERFAZ.....	5
2.1. MODO INTERACTIVO	5
2.2. MODO DE PROCESAMIENTO POR LOTES	5
2.3. FUNCIONES EMPLEADAS	5
2.3.1. Run_commands.....	6
2.3.2. Run_program.....	6
2.3.3. Clean_command	6
2.3.4. Funciones auxiliares	8
3. FUNCIONES	10
3.1. Es_PRIMO.....	10
3.2. LISTA_PRIMOS	11
3.3. FACTORIZAR	11
3.4. MCD.....	12
3.4.1. Algoritmo de Euclides.....	12
3.4.2. Bezout	13
3.5. MCD_N	13
3.5.1. Mcd con n elementos	13
3.5.2. Teorema de Bezout para n elementos	14
3.6. COPRIMOS.....	15
3.7. POTENCIA_MOD_P	15
3.8. INVERSA_MOD_P	17
3.9. EULER	17
3.10. LEGENDRE	18
3.11. RESOLVER_SISTEMA_CONGRUENCIAS	19
3.11.1. Reolver_lineal	20
3.11.2. Tcr_dos	21
3.11.3. Mod	21
3.12. CARMICHAEL.....	21
4. BIBLIOGRAFÍA.....	24

1. INTRODUCCIÓN

Este documento expone el trabajo de Gabriela Romero Martín y Candela Tejedo Raga, alumnas de la Escuela Técnica Superior de Ingeniería (ICAI) de la Universidad Pontificia Comillas. Dicho trabajo lleva como título “IMAT-LAB” y se ha realizado dentro de la asignatura de “Matemática Discreta” de 2º de iMAT.

La aritmética modular es un conjunto de métodos que nos permiten resolver ciertos problemas sobre números enteros, fue introducida en 1801 por Carl Friedrich Gauss en su libro *Disquisitiones Arithmeticae*. En esta práctica se nos pide desarrollar una librería y una interfaz llamada IMAT-LAB que resuelva problemas de este tipo empleando distintos algoritmos. Esta librería dispone de un total de 12 comandos diferentes.

- `primo(n)`: valora si un número es primo o no
- `primos(a, b)`: devuelve los primos que hay en el intervalo $[a, b]$
- `factorizar(n)`: realiza la descomposición factorial de un número
- `mcd`: realiza el máximo común divisor entre 1 o más números
- `coprimos(a, b)`: valora si dos números son coprimos entre sí
- `inv(n,p)`: realiza la inversa de un número n con módulo p
- `euler(n)`: calcula la función Euler $\varphi(n)$
- `legendre(n, p)`: aplica el algoritmo de legendre a $n \pmod{p}$
- `resolverSistema`: resuelve un sistema de n ecuaciones de congruencias
- `carmichael(n)`: le aplica la conjetura de carmichael a un número entero

Del mismo modo, para la resolución de varios de estos comandos se han necesitado las siguientes funciones auxiliares:

- `bezout / bezout_n`: calcula el máximo común divisor empleando el algoritmo de bezout.
- `mod`: calcula el módulo de dos números enteros
- `tcr_dos`: resuelve dos ecuaciones de congruencias por el Teorema Chino del Resto
- `resolver_lineal`: resuelve una ecuación congruente $ax \equiv b \pmod{p}$.

Además, se ha implementado una gestión de errores en la entrada de comandos. Se ha valorado si el comando introducido existe y los argumentos de este se han mandado de forma adecuada y si, en el caso de mandarlos, los ficheros introducidos existen. Si este no es el caso se imprime un mensaje especificando el error.

El programa necesita de tres ficheros de Python. `Modular.py`, donde se encuentran todas las funciones que resuelven el comando recibido, `imatlab.py`, la interfaz donde se gestiona la entrada y salida de datos y resultados y `errores.py` donde se encuentran las excepciones empleadas a lo largo del programa.

Todo el código se ha realizado buscando la funcionalidad correcta y la eficiencia de todos los comandos ayudándonos en ocasiones de fuentes externas citadas en la bibliografía.

Este documento se compone de una introducción, de una explicación del funcionamiento de la inetrfaz (imatlab.py) y de las funciones empleadas (modular.py) justificando las decisiones tomadas a lo largo del protyecto, un apartado de conclusiones y una bibliografía citando las fuentes utilizadas.

2. INTERFAZ

Para realizar la interfaz se ha empezado por distinguir entre los dos casos de ejecución posibles, el modo de procesamiento por lotes, si se reciben dos ficheros, uno de entrada y otro de salida por los argumentos y, el modo interactivo, si no se reciben argumentos se le propondrá al usuario escribir un comando a resolver. Es necesario comentar que, en el caso de recibir más de dos argumentos por la terminal se ha decidido pasar automáticamente al modo interactivo. Se decidirá el modo de ejecución al principio del programa empleando un `if/else` y el comando `sys.srgv`. En ambos casos los comandos se ejecutan empleando la misma estrategia con la particularidad de que en el modo de procesamiento por lotes previamente se debe realizar una lectura de ficheros para obtener el comando que se quiere realizar.

La estructura es simple, una vez se tiene la instrucción a realizar se pasa por un proceso de validación, en el caso en el que la instrucción no tenga en formato adecuado se gestionará el error (dicha gestión se explicará más adelante). Una vez hecho esto una función auxiliar llamará a la función guardada en el fichero `modular.py` que obtendrá la solución empleando un determinado algoritmo. La solución se imprimirá en el formato adecuado en la pantalla o en un fichero de salida en función del modo en el que nos encontremos.

2.1. Modo interactivo

En este caso se ha recibido el comando que quiere ejecutar el usuario. Se entrará en un bucle `while` que, empleando la función `run_program` (explicada más adelante) ejecuta dicho comando y, una vez haber impreso la respuesta en la terminal ofrece la posibilidad de introducir otro comando. Este bucle se detendrá cuando el usuario pulse enter.

2.2. Modo de procesamiento por lotes

En este se han recibido dos ficheros válidos por argumentos por tanto entramos en modo automático. El programa ejecutará los comandos escritos en el fichero (denominado como `fin`) y escribirá la salida en otro fichero (denominado como `fout`). para ello se empleará la función `run_commands` (explicada más adelante). El programa finalizará una vez haya terminado de leer el fichero de entrada (`fin`).

2.3. Funciones empleadas

Para ejecutar el programa de manera óptima se han empleado ciertas funciones guardadas en el fichero principal, en `imatlab.py`. Estas son las que procesan la información recibida y llaman a las funciones del fichero `modular.py`.

2.3.1. Run_commands

Esta función toma dos ficheros ya abiertos, uno de lectura (fin) y otro de escritura (fout). Lee el contenido de fin línea por línea mandándolo a run_commands que le devuelve el resultado, dicho resultado lo escribe en el fichero de salida (fout). Esta función se empeña únicamente si el programa se encuentra modo de procesamiento por lotes y se ejecuta tantas veces como líneas haya en el fichero de entrada.

2.3.2. Run_program

Esta Función será el núcleo de la interfaz, recibe el comando a ejecutar en modo string y devuelve su resultado. Para ello, dentro de esta definiremos un diccionario, dic_comandos, que establecerá la relación entre estos strings y sus funciones asociadas. La estructura de este será la siguiente: la clave del diccionario es el string que se debe recibir por pantalla y su valor asociado es una lista donde en la primera posición (posición 0 en indexación de Python) está la función de modular.py asociada, en la segunda posición el número de argumentos que debe recibir dicha función (en el caso de que la función no tenga un número predeterminado de argumentos en esta posición encontraremos un None) y la última posición de la lista es el nombre de la función auxiliar almacenada en el main que emplearemos para llamar a la función de modular.py.

Ejemplo: para el comando primo(n) su item del diccionario será: 'primo': [f.es_primo, 1, run_bool]

Desde aquí se llamará a clean_command que validará el comando del usuario y si el formato es adecuado devolverá las funciones y los argumentos que se emplean para su ejecución, si la estructura del comando no es válida gestiona el error. Después, llamará a la función auxiliar que le devolverá la solución buscada.

2.3.3. Clean_command

Función que limpia y valida los comandos recibidos por pantalla encontrando la función de modular.py que se corresponde con la que pide el usuario empleando expresiones regulares en lenguaje Regex con el fin de volver la búsqueda más eficiente. En el caso de que el usuario haya introducido el comando mal se lanzará un NOPErrror.

El comando debe tener la estructura comando(argumentos) por lo tanto nos interesa crear una expresión que se divida en dos grupos, el grupo 1 es el comando y el 2 los argumentos. Se ha elegido la siguiente expresión:

$$"([a-zA-Z_]+)\((.*)\)"$$

En cuanto al grupo 1, una vez hallado se validará si este es una clave de dic_comandos. De no serlo se devolverá error y de serlo se crearán las variables comando_nuevo (la función de modular.py asociada) y llamar comando (la función auxiliar necesaria para la ejecución).

Es necesario comentar que para la validación de argumentos se ha optado por una estrategia específica. En Esta expresión empleamos lo que se conoce como 'lazy mode' (.?). A lo largo de la realización de las funciones se observó que todas las funciones

recibían argumentos similares sino iguales, números. Sin embargo, una función se diferenciaba del resto, `resolverSistema`, que debía recibir tres listas. Debido a esto, con el fin de simplificar el código de `clean_command` lo máximo posible se decidió aceptar inicialmente todo lo que el usuario ponga dentro del paréntesis y luego distinguir entre dos casos, si la función es `resolverSistema` y si no lo es.

Si lo es buscamos que los argumentos sigan la estructura $[a_1; b_1; p_1], [a_2; b_2; p_2], \dots, [a_n; b_n; p_n]$, y los transformaremos en una lista que contenga tres listas: $[[a_1, a_2, \dots, a_n], [b_1, b_2, \dots, b_n], [p_1, p_2, \dots, p_n]]$. Para definir esta estructura hemos empleado la siguiente expresión regular:

$$\text{"\[(-?\d+);(-?\d+);(-?\d+)\]\}"}$$

Esta expresión la acompañamos del comando `re.findall()` que devuelve todas las apariciones en la cadena de caracteres de dicha expresión guardándolas por separado en una lista (`match_r_sist`). Para crear la lista solución debemos extraer el primer, segundo y último elemento de cada lista y guardarlos en tres listas respectivamente. Para ello utilizaremos tres comandos de Python distintos: `map`, `zip` y `*`.

El comando `map` convierte todos los elementos de una lista en un tipo de carácter específico, en este caso en particular se usará para convertir a `int` y después para convertir a lista. Además, usaremos el comando `*` para expandir los objetos creados con `map` con el fin de poder continuar manipulándolos sencillamente. Por otro lado, el comando `zip` nos resultará muy útil en esta ocasión ya que combina varios iterables del mismo tipo en paralelo, es decir, tomará todas las listas recibidas por argumentos, podemos pensar en ellas como una matriz de $n \times 3$, y traspone dicha matriz dejando una de $3 \times n$, que es justo lo que necesitamos.

En resumen, tomaremos `match_r_list` y convertiremos su contenido en tipo `int`, expandiremos ese resultado y traspondremos, finalmente convertiremos el resultado en una lista de listas, lista que contiene las tres listas solución.

Ejemplo: $[[2; 3; 4], [4; 8; 9], \dots, [1; 7; 6]] \rightarrow [[2, 4, \dots, 1], [3, 8, \dots, 7], [4, 9, \dots, 1]]$

Línea de Código:

```
lista_argumentos = list(map(list, zip(*(map(int,num) for num in match_r_sist)))).
```

Finalmente, si esta lista acaba estando vacía o el tamaño de las tres listas no coinciden con el número de listas que mandó el usuario se devolverá un error.

Por otra parte, si los argumentos que buscamos son números sueltos emplearemos la siguiente expresión regular:

$$\text{"-?\d+"}$$

Esta irá acompañada de un `re.findall()` con el que obtendremos una lista con los distintos argumentos (en tipo `string`). Le aplicaremos `map` a la lista para convertir su contenido a `int`.

En ambos casos la función devuelve lo mismo, una tupla con la función de modular.py a ejecutar (comando_nuevo), la función auxiliar (llamar_comando) y la lista de los argumentos (lista_argumentos).

Empleando expresiones regulares creamos el patrón de la estructura que debe tener el input de pantalla (comando(Argumentos)). Esta expresión se divide en dos grupos. El primero corresponde al comando que se quiere realizar (más adelante se validará si este es válido, si tiene una función de modular.py asociada) y el segundo a todo lo que este dentro de los parentesis de forma perezosa (lazy). Más adelante se validará si lo que se encuentra dentro del paréntesis tiene el formato adecuado haciendo uso del método findall que, junto a la expresión “-?[0-9]+” que nos creará una lista solo con los números (todavía serán tipo int, eso se cambiará más adelante) que encuentre en el grupo 2 (el grupo de los argumentos).

2.3.4. Funciones auxiliares

Las siguientes funciones son funciones auxiliares que se han empleado para llamar a las distintas funciones de modular.py. Con el fin de hacer el programa lo más eficiente posible y habiéndonos percatado de que muchas de las funciones se relacionan entre ellas en sus salidas y entradas se han programado 7 funciones auxiliares.

Run_bool: esta función llamará a todas las funciones de modular.py que devuelven un booleano: es_primo y coprimos. Devolverá ‘Sí’ si la función si la salida de la función de modular.py es True y ‘No’ si es False.

Run_lprimos: esta función llamará a lista_primos de modular.py y según la salida de esta devolverá: la lista de números primos pedida (en formato string sin los corchetes) o, en el caso de que esta lista este vacía, ‘NE’.

Run_factors: esta función llamará a factorizar de modular.py y según la salida de esta devolverá: el diccionario de la descomposición factorial del número (en formato string sin las llaves) o, en el caso de que el diccionario este vacío 0.

Run_mcd: esta función llamará a mcd si recibe estrictamente dos argumentos, dos números o a mcd_n si recibe cualquier otro número de argumentos. Devolverá el resultado obtenido de la función empleada.

Run_mod_p: esta función llamará a todas las funciones de modular.py que devuelvan un número (run_mod_p devuelve dicho número) pero que, además, en algunos casos puedan devolver un error de tipo ZeroDivisionError (run_mod_p devolverá ‘NE’) o NOPEError (run_mod_p devolverá ‘NOP’): pow, inv, euler, legendre.

Run_resSist: esta función llamará a resolver_sistema_congruencias de modular.py. Esta devolverá una tupla con el resultado del sistema (en este caso run_resSist devuelve r (mod m) en formato string) o, en caso de que la función de modular.py devuelva un error, ya sea de tipo ValueError o IncompatibleEquationError, se devolverá un string

explicando el porque del error (por ejemplo, si el sistema es incompatible o no tiene solución).

3. FUNCIONES

3.1. Es_primo

Un número primo es un entero positivo p que únicamente tiene dos divisores positivos distintos: 1 y él mismo

Si n no es primo, es compuesto, y existen enteros $a, b > 1$ tales que $n = ab$. De estos dos factores, al menos uno tiene que ser de la forma $x \leq \sqrt{n}$.

Demostración por reducción al absurdo:

Suponemos que ambos factores fuesen $\geq \sqrt{n}$. Entonces,

$$ab > \sqrt{n} \sqrt{n} = n$$

Pero esto es imposible porque $ab = n \therefore$ no pueden ser los dos $\geq \sqrt{n}$

Por lo que, si busco un divisor de n , basta con comprobar si existe alguno hasta $\sqrt{n}$; si ninguno divide entonces no existe ninguna pareja $ab = n$ por lo que n será primo.

Aplicación de esta teoría en la función $es_primo(n: int) \rightarrow bool$:

1. Descarto los casos triviales ($n < 2$) y acepto $n = 2$ como primo por definición
2. Descarto los números pares ya que siempre se podrán dividir por 2 a parte de por 1 y ellos mismos (siempre son compuestos)
3. Iteramos desde 3 hasta $\lfloor \sqrt{n} \rfloor$ porque, como hemos demostrado antes, si n tuviera algún divisor, existiría uno $\leq \sqrt{n}$
4. El bucle avanza de dos en dos porque sabemos que n no es par
5. En cuanto se encuentra un divisor, devuelve False (no es primo); de lo contrario, devuelve True al salir del bucle una vez de haber probado todos los enteros hasta $\lfloor \sqrt{n} \rfloor$

Optimizaciones:

- Al acotar el recorrido del bucle por $\sqrt{n}$ se reduce la complejidad de $O(n)$ a $O(\sqrt{n})$
- Después de haber aceptado $n = 2$ como primo recorre solo impares por lo que hace la mitad de las divisiones
- Devuelve False en cuanto detecta un divisor por lo que evito trabajo innecesario

3.2. Lista_primos

Para obtener una lista de los primos en un intervalo semiabierto $[a, b)$, nuestra estrategia será recorrer el intervalo y, para cada número x , decidir si es primo usando la función anterior *es_primo()*.

Aplicando este razonamiento a la función *lista_primos(a: int, b: int) → list[int]*:

1. Inicializamos una lista vacía que devolveremos al principio de la función si $a \geq b$ o al final de la función si no existiese ningún primo en ese intervalo
2. Recorremos todos los números del intervalo, empezando desde el primer primo (2) o desde a , dependiendo de cual sea más grande
3. Se le aplica la función *es_primo()* a cada número y en caso de que lo sea, se concatena a la lista
4. Devolvemos la lista de todos los primos en el intervalo $[a, b)$

Optimizaciones:

- Salida inmediata en caso de que los límites del intervalo estén invertidos o sean vacíos
- Acortamos el recorrido del bucle al dominio relevante: nunca un número ≤ 2
- Todas las optimizaciones implementadas en *es_primo()*

3.3. Factorizar

El teorema Fundamental de la Aritmética establece que todo número entero mayor que 1 es o bien un número primo, o puede ser expresado de manera única como un producto de números primos:

$$n = \prod_{i=1}^k p_i e^i$$

La función factorizar devolverá un diccionario donde las claves son los primos que dividen a n y los valores sus exponentes.

Como hemos visto en el apartado 1, si n es compuesto, existe un primo $p \leq \sqrt{n}$ que lo divide. Por tanto, para factorizar bastará otra vez con probar divisores hasta $\sqrt{n}$; y si tras eso queda un $x > 1$, ese x debe de ser primo.

Por otra parte, todo primo ≥ 5 no es múltiplo ni de 2 ni de 3, por lo tanto, su resto módulo 6 será 1 o 5, en otras palabras, todos los números que quedan por considerar serán de la forma $6k \pm 1, k \in \mathbb{N}$. Habernos dado cuenta de esto nos permite eliminar todos los demás ‘candidatos’ a primo mejorando así la eficiencia del algoritmo.

Aplicando este razonamiento a la función $factorizar(n: int) \rightarrow dict[int, int]$:

Dentro de la función se creará un diccionario vacío. Aquí se almacenarán los resultados. Las claves de este serán todos los primos, p_i divisores del número y sus valores asociados serán sus exponentes, e_i .

1. Empezamos validando que la entrada $\notin \{-1, 0, 1\}$ y si lo está, se devuelve el diccionario vacío.
2. Extrae todas las potencias de 2 y de 3 de tal manera que x es coprimo de 6.
3. Entramos en un bucle que solo recorre los números de la forma $6k \pm 1$ alternando entre saltos de 2 y 4 unidades para comparar únicamente con números coprimos a 6.
4. Si el resto al final del bucle es > 1 , x es primo y lo añado al diccionario con exponente 1

Optimizaciones:

- Acotamos el recorrido del bucle con $while\ p_divisor * p_divisor \leq x$: y en vez de recorrerlo hasta n lo recorreremos solo hasta $\sqrt{n}$, usando una vez más la justificación del apartado 1: si n es compuesto, existe un primo $< \sqrt{n}$ que lo divide
- Al quitar primero todos los factores de 2 y de 3, el x restante es más pequeño y coprimo con 6, lo que permite después saltar los múltiplos de 2 y 3 y no tener que recorrer el bucle para tanta cantidad de números
- Cada vez que encontramos un primo factor de x , dividimos por ese primo. Por lo que cada división reduce x y la condición de parada ($\sqrt{n}$) va bajando; por lo que el bucle externo acabará antes de alcanzar el límite inicial.

3.4. Mcd

3.4.1. Algoritmo de Euclides

Sean $a, b \in \mathbb{Z}$, $mcd(a, b)$ es el mayor común divisor de a y b . El algoritmo de Euclides visto en clase se basa en un principio muy sencillo, el máximo común divisor de dos números no cambia si cambiamos el mayor por el resto de la división de este entre el menor. Esto nos permite calcular el mcd con la siguiente recursión: $mcd(b, b \bmod a)$ hasta que el par toma la forma $(d, 0)$ para algún d , siendo $mcd(a, b) = d$.

Aplicando esta teoría a la función $mcd(a: int, b: int) \rightarrow int$:

1. $(a, b) \leftarrow (b, a \bmod b)$ hasta que $b = 0$ y luego devuelvo $|a|$ como mcd

Optimizaciones:

- Este algoritmo tiene una complejidad de $O(\log(\min(a, b)))$ ya de por sí y por lo tanto se ha decidido no optimizarlo más.

3.4.2. Bezout

El teorema de Bezout establece que para dos polinomios su máximo común divisor puede ser expresado como una combinación lineal de dichos polinomios. En otras palabras, el teorema dice que existen $x, y \in \mathbb{Z}$ tal que $ax + by = \text{mcd}(a, b)$. Este, también llamado el algoritmo de Euclides extendido hace lo mismo que el algoritmo de Euclides Comentado en el apartado anterior, pero recordando en cada paso como se obtiene el resto como combinación lineal de a y b . Esa combinación lineal tiene la forma: $r = ax + by$ con $x, y \in \mathbb{Z}$. Y al final, el resto antes del 0 es el $\text{mcd}(a, b)$ y los coeficientes que habían arrastrados en ese paso son x, y tal que $ax + by = \text{mcd}(a, b)$.

En el algoritmo, q es el número entero que multiplicado por el divisor (r) resulta en un producto $\leq$ al que estamos dividiendo (r_{antes}) y vamos a ir restándole qr a r_{antes} hasta llegar al resto 0; pero a la vez, le haremos lo mismo a x, y para ir actualizando los coeficientes también (podemos pensar en estos dos números como vectores pertenecientes a $\mathbb{R}^2$ sabiendo que su base canónica $\{(1, 0), (0, 1)\}$ se corresponde con $\{(a, 0), (0, b)\}$). Por lo que en cada iteración tendremos dos ecuaciones en memoria:

$$r_{\text{antes}} = ax_{\text{antes}} + by_{\text{antes}}, r = ax + by$$

Aplicando esta idea a la función $\text{bezout}(a: \text{int}, b: \text{int}) \rightarrow \text{tuple}[\text{int}, \text{int}, \text{int}]$:

1. Inicializamos los coeficientes como $a = 1a + 0b$, $b = 0a + 1b$ (las coordenadas a papel)
2. Hasta que el resto es 0 (mcd), actualizo las 3 variables, restándoles un múltiplo del cociente de restos
3. Normalizo porque al final $r_{\text{antes}} = \text{mcd}(|a|, |b|)$ y queremos $= \text{mcd}(a, b)$

3.5. Mcd_n

3.5.1. Mcd con n elementos

El máximo común divisor puede calcularse entre más de dos números. Este cálculo se define como: $\text{mcd}(a_1, a_2, \dots, a_n) = \text{mcd}(\text{mcd}(\dots \text{mcd}(a_1, a_2), a_3), \dots, a_n)$. Es decir, calcula el máximo común divisor entre todos los números de la lista dos a dos hasta llegar al resultado final.

Aplicando esta idea la función $\text{mcd}_n(nlist: \text{list}[\text{int}]) \rightarrow \text{int}$:

1. Empezamos creando la variable g y asignándola el valor 0.
2. Entramos en un bucle que recorre la lista de argumentos uno a uno calculando el $\text{mcd}(g, a)$ siendo a el número en cuestión. El resultado de este para a ser el nuevo valor de g . Aquí cabe destacar que esta función también acepta realizar el mcd de un único número ya que $\text{mcd}(0, a) = a, \forall a \in \mathbb{N}$.
3. En el interior de este bucle se implementa una condición de salida adicional. En el momento en el que $g = 1$, el mínimo divisor, se devuelve ese valor y se sale de la función.
4. Una vez se salga del bucle el mcd será el valor de g .

Este algoritmo, teniendo en cuenta que emplea el algoritmo de Euclides explicado en el apartado anterior, tendrá una complejidad de $O(n \log(\min(a, b)))$ siendo n la longitud de la lista mandada.

3.5.2. Teorema de Bezout para n elementos

Teniendo en cuenta lo que enuncia el teorema de bezout, (explicado en el apartado anterior) empleamos dicho teorema expandiéndolo para n números enteros, $a_1, a_2, \dots, a_n$ y por lo tanto el resultado se extiende de tal modo que:

$$\exists x_1, x_2, \dots, x_n \text{ tal que } a_1 x_1 + a_2 x_2 + \dots + a_n x_n = \text{mcd}(a_1, a_2, \dots, a_n).$$

El objetivo de este algoritmo será por lo tanto encontrar los enteros $x_1, x_2, \dots, x_n$ que cumplan la ecuación:

$$d = \sum_{j=1}^N a_j x_j$$

donde $d = \text{mcd}(a_1, a_2, \dots, a_n)$.

La idea detrás de este algoritmo es ir aplicando iterativamente el teorema de bezout a cada par de números construyendo los coeficientes en el proceso. Ya que el mcd de varios números se calcula también de manera iterada, cada paso añade un coeficiente nuevo a la ecuación y al mismo tiempo ajusta los anteriores manteniendo la linealidad de esta. Demostremos ahora que la identidad de bezout se cumple para n números por inducción:

Primero, para unos números $a = a_1, a_2, \dots, a_n, a_{(n+1)}$ aplicamos el teorema de Bezout a los dos últimos:

$$d_n = x_n a_n + x_{(n+1)} a_{(n+1)}$$

Después elaboramos nuestra hipótesis inductiva, suponemos que se cumple para

$$n = k \Rightarrow \text{mcd}(a_1, a_2, \dots, a_k) = a_1 x_1 + a_2 x_2 + \dots + a_k x_k$$

Ahora exploraremos si el teorema se cumple para $n = k + 1$:

Sea $d_k = \text{mcd}(d_k, a_k)$ por hipótesis inductiva:

$$d_k = a_1 x_1 + \dots + a_k x_k.$$

Ahora tomamos el siguiente número $a_{(k+1)}$ y sea $d_{(k+1)} = \text{mcd}(d_k, a_{(k+1)})$.

Por Bezout para dos números se cumple que

$$d_{(k+1)} = v d_k + w a_{(k+1)}, s, t \in \mathbb{Z}.$$

Sustituyendo d_k obtenemos:

$$d_{(k+1)} = v(a_1 x_1 + \dots + a_k x_k) + t a_{(k+1)}$$

Si reordenamos los términos nos queda:

$$d_{(k+1)} = (sx_1)a_1 + \dots + (sx_k)a_k + ta_{(k+1)}$$

Por lo tanto, la hipótesis se cumple también para $n = k + 1$.

Aplicando esta idea la función $bezout_n(nlist: list[int]) \rightarrow tuple[int, list[int]]$:

1. Evaluamos el caso base, si todos los números de la lista son 0 su $mcd = 0$.
2. Inicializamos el algoritmo con el primer número de la lista, tomamos $d = |a_1|$ y por lo tanto el primer coeficiente será $x_1 \pm 1$ ya que cumple que $a_1 x_1 = d$
3. Entramos en un bucle que recorre la lista y va resolviendo por bezour de dos de manera iterativa
4. Terminamos por corregir los coeficientes de los números sustituyendo d por la identidad de Bezout calculada.

Optimizaciones:

- Cabe destacar que al igual que en el algoritmo de Bezout se ha tomado la decisión de, en el momento en el que $d = 1$ quiere decir que dos números de la lista son coprimos y por lo tanto el resultado de la operación debe de ser 1. Esto evita realizar cálculos innecesarios haciendo más eficiente el proceso.

3.6. Coprimos

Sean $a, b \in \mathbb{Z}$, a y b son coprimos si su máximo común divisor es 1:

$$mcd(a, b) = 1$$

Aplicando esta definición a nuestra función $coprimos(a: int, b: int) \rightarrow bool$:

1. Llamo a la función $mcd()$ que hace el algoritmo de Euclides y comparo el resultado con 1
2. Si el $mcd = 1$ devuelve True

Optimización:

- La función $coprimos()$ tiene complejidad $O(1)$ además de trabajar con una función ($mcd()$) también óptima.

3.7. Potencia_mod_p

Decimos que un número es congruente a otro con módulo p ($x \equiv n(mod p)$) si se cumple que $p \mid x - n$. Sabiendo esto, en este caso nos gustaría calcular x siendo n un número elevado a un cierto exponente b^{exp} . Para ello emplearemos un concepto muy útil en el campo de la criptografía, la exponenciación modular.

Es necesario resaltar que esta también funciona si el exponente negativo, en este caso simplemente se deberá calcular la inversa teniendo en cuenta que: b^e con $e < 0 = b^{-1}b^{|e|}$.

Este algoritmo emplea algo conocido como la exponenciación binaria, un método empleado para calcular grandes potencias enteras de un número. Lo primero que se debe hacer es escribir el exponente recibido en notación binaria. Escribimos e como:

$$e = \sum_{i=0}^{n-1} b_i 2^i$$

Donde a_i representa los dígitos binarios, los bits, que pueden ser 0 o 1 y 2^i es la potencia de 2 correspondiente a la posición de dicho bit. Por lo tanto, la longitud de e será de n bits. Sabiendo esto podemos reescribir b^e como:

$$b^e = b^{\sum_{i=0}^{n-1} a_i 2^i} = \prod_{i=0}^{n-1} b^{a_i 2^i}$$

Aplicando esta fórmula a nuestro problema nos queda que:

$$c \equiv \prod_{i=0}^{n-1} b^{a_i 2^i} \pmod{m}$$

Aplicando esta idea a nuestro algoritmo `potencia_mod_p(base: int, exp: int, p: int) → int`:

1. Comenzamos por los casos base. Si alguno el módulo recibido por argumentos es 0 devuelve error al igual que si se desea calcular $0^0 \pmod{p}$.
2. Si la base es menor que 0 se actualiza a un número positivo sumándole el valor del módulo.
3. Si el exponente es negativo se actualizarán el y la base empleando la inversa.
4. El resultado comienza con valor 1. Entramos en el bucle. Aquí se evaluará si el último bit del exponente binario es 0 o 1, esto es equivalente a evaluar si el número del exponente es par o impar.
5. Si este es impar el último bit tendrá valor 1 y, según la fórmula enunciada anteriormente debemos actualizar el resultado de tal forma que el nuevo resultado será el anterior multiplicado por la base con módulo p , $result = (result * base) \pmod{p}$. Después se actualizará la base tal y como dice la fórmula de tal modo que $base = (base * base) \pmod{p}$. Si el valor asociado al último bit es 0 simplemente se actualizará la base.
6. Por último, actualizamos el valor del exponente eliminando el bit que acaba de ser procesado. Por lo tanto, el exponente actualizado será el resultado de aplicarle la función suelo a la división del exponente entre 2, en otras palabras, $e = \lfloor \frac{e}{2} \rfloor$

El bucle se detendrá cuando el valor del exponente sea menor que 0.

Optimizaciones:

La ventaja que tiene esta fórmula es que en cada iteración del bucle se eliminará un bit del exponente lo que, trabajando con números grandes, baja notablemente la complejidad del algoritmo, de $O(e)$ a $O(\log(e))$.

3.8. Inversa_mod_p

En aritmética modular decimos que un número tiene inversa si $\exists x : ax \equiv 1 \pmod{m}$. En otras palabras, se debe de cumplir que $m \mid ax - 1$.

Para calcular la inversa nos resulta útil emplear el algoritmo de Bezout para calcular el $mcd(a, m)$. Este siempre debe valer 1 ya que una condición necesaria para la existencia de la inversa es que los dos números sean coprimos.

Aplicando esta definición a la función $inversa_mod_p(n: int, p: int) \rightarrow int$:

1. Primero descartamos los casos imposibles, cuando el módulo es 0 o cuando es menor que 1, en ambos casos el algoritmo devolverá un error que se gestionará en la interfaz.
2. Aplicamos el algoritmo de bezout de manera que $mcd(n, p) = x_0n + y_0p$. Puesto que $mcd(n, p) = 1 \Rightarrow 1 = x_0n + y_0p \Rightarrow m \mid x_0n - 1$
3. Si el $mcd \neq 1$ se devolverá un error ya que la inversa no existe, de lo contrario devuelve x_0 normalizado.

3.9. Euler

Sea $n \geq 1 : n \in \mathbb{Z}$. Definimos la función φ de Euler como la función $\varphi: \mathbb{N} \rightarrow \mathbb{N}$ que, para cada n , devuelve la cantidad de enteros positivos menores que n que son coprimos con él, En otras palabras, la función de Euler es aquella que cumple:

$$\varphi(n) = \# \{k \in \mathbb{N} \mid k \leq n, (k, n) = 1\}$$

Es importante saber que esta función cumple 4 propiedades que son las que nos han permitido desarrollar de manera óptima la función.

- Si p es primo se cumple que $\varphi(p) = p - 1$
- $mcd(1,1) = 1, n$ es coprimo consigo mismo y por tanto $\varphi(1) = 1$.
- Si $mcd(a, b) = 1 \Rightarrow \varphi(ab) = \varphi(a)\varphi(b)$.
- Si p es primo se cumple que $\varphi(p^k) = (p - 1)p^{k-1}$.

Por lo tanto, para hallar $\varphi(n)$ para un cierto $n \in \mathbb{N}$ es interesante ver el número como un productorio de números primos. En otras palabras, $n = \prod p_i^{e_i}$ tal que p es primo. De este modo podemos calcular $\varphi(n) = \varphi(p_i^{e_i})$, aplicando la tercera propiedad enunciada anteriormente, ya que p_i es primo $\forall i \in \mathbb{N}$, se cumple que $mcd(p_i^{e_i}, p_j^{e_j}) = 1 \forall i, j \in \mathbb{N}$ y, en consecuencia: $\varphi(n) = \prod \varphi(p_i^{e_i})$ y por la cuarta propiedad vemos que $\varphi(p^e) = (p - 1)p^{e-1} \Rightarrow \varphi(n) = \prod (p_i - 1)p_i^{e_i-1}$.

Aplicando esta definición a la función $euler(n: int) \rightarrow int$:

1. Empezamos distinguiendo algunos casos base. Por convenio $\varphi(0) = 0$, devolvemos este resultado. Del mismo modo, se devuelve un error en el caso de que $n \notin \mathbb{N}$.
2. Del mismo modo si tenemos $n = 1$ puesto que $\varphi(1) = 1$ (por la segunda propiedad enunciada anteriormente), devolvemos el valor automáticamente.
3. Una vez excluidos estos casos empleamos la función de factorizar para obtener los valores primos y sus exponentes del número divisores del número.
4. Comenzamos con $\phi = 1$ y entramos en un bucle que aplica las propiedades comentadas anteriormente calculando el productorio solución.

3.10. Legendre

En teoría de números el símbolo de Legendre es una función multiplicativa, se utiliza para determinar el carácter cuadrático de un cierto $x \pmod{p}$, en otras palabras, se desea ver si dicho número tiene residuo cuadrático o no. Decimos que un número módulo m tiene residuo cuadrático r cuando se cumple que $x^2 \equiv r \pmod{m}$. El algoritmo de Legendre toma un número entero n y un primo p , en el estudio de residuos cuadráticos solemos limitarnos al caso en el que el módulo es primo ya que estos tienen comportamientos mucho más sencillos y muchas de las propiedades de los residuos cuadráticos pueden derivarse a este ámbito empleando el teorema chino del resto.

Este algoritmo tiene tres posibles salidas, 0, 1, -1. Denotamos el símbolo de Legendre por

$$\left(\frac{n}{p}\right) \text{ y se define como: } \left(\frac{n}{p}\right) = \begin{cases} 0 & \text{si } p \text{ es divisor de } a \\ 1 & \text{si } a \text{ es residuo cuadrático módulo } p \\ -1 & \text{si } a \text{ es no residuo cuadrático módulo } p \end{cases}$$

En este algoritmo p debe ser estrictamente un número primo ya que la definición usa el hecho de que el conjunto de todos los primos con módulo m forman un cuerpo, forman un anillo conmutativo y unitario donde todos sus elementos invertibles. En el momento en el que quisiéramos emplear un número no primo dejaríamos de tener un cuerpo ya que habría números cuya inversa no existe.

Por otro lado, el teorema también enuncia que p debe ser un primo distinto de 2 ya que su símbolo de Legendre resulta trivial, sea $x^2 \equiv r \pmod{2}$, $r = 0$ o $r = 1$. En otras palabras, el símbolo de Legendre sería:

$$\left(\frac{n}{2}\right) = 1 \text{ si } n \text{ es impar puesto que } n^2 \equiv 1 \pmod{2},$$

$$\left(\frac{n}{2}\right) = 0 \text{ si } n \text{ es par puesto que } n^2 \equiv 0 \pmod{2}$$

Un método común para determinar si un número entero es un residuo cuadrático módulo primo es emplear el criterio de Euler. Este dice lo siguiente: sea p un primo impar y n un entero coprimo de este entonces:

$$n^{\frac{p-1}{2}} \equiv \begin{cases} 1 \pmod{p} & \text{si } \exists x \in \mathbb{N} : a \equiv x^2 \pmod{p} \\ -1 \pmod{p} & \text{si } \nexists x \end{cases}$$

Esto se puede reformular utilizando el símbolo de Legendre de tal manera que:

$$\left(\frac{n}{p}\right) = n^{\frac{p-1}{2}} \pmod{p}$$

Aplicando esta definición a la función $legendre(n: int, p: int) \rightarrow int$:

1. Comenzamos validando el módulo p , este debe ser distinto de 0 y debe ser primo. Si no cumple alguna de estas se devuelve error.
2. Normalizamos el módulo, si este es 0 p es divisor de n y por lo tanto devolvemos 0. Después, aplicamos el criterio de Euler mencionado anteriormente.
3. Si el resultado de este es 1 entonces el número si es residuo cuadrático con módulo p y por lo tanto se devuelve 1. De lo contrario devolvemos -1.

3.11. Resolver_sistema_congruencias

Dados $a \in \mathbb{Z}$ y $m > 0$, la división entera es $a = qm + r$ con $0 \leq r < m$ y definimos el resto r como $a \bmod m = r$.

Decimos que a es congruente con b módulo m : $a \equiv b \pmod{m}$ si y sólo si $m|(a - b)$.

Ecuaciones lineales de congruencias $ax \equiv b \pmod{m}$

Sea $d = \text{mcd}(a, m)$

- La congruencia tiene solución si y sólo si $d|b$
- Si $d|b$ entonces al dividir por d se obtiene una congruencia equivalente:

$$\frac{a}{d}x \equiv \frac{b}{d} \pmod{\frac{m}{d}}, \text{ con } \text{mcd}\left(\frac{a}{d}, \frac{m}{d}\right) = 1$$

Sean $a_1 = \frac{a}{d}$, $b_1 = \frac{b}{d}$, $m_1 = \frac{m}{d}$

- Como $\text{mcd}(a_1, m_1) = 1$, hay una única solución módulo m_1 : existe x tal que

$$a_1x \equiv b_1 \pmod{m_1} \rightarrow x \equiv a_1^{-1}b_1 \pmod{m_1}$$

Y hay d soluciones distintas módulo m , separadas por saltos de $\frac{m}{d}$.

Teorema Chino del Resto

El teorema chino del resto dice que sean $a_1, \dots, a_n, p_1, \dots, p_n \in \mathbb{Z}$ con $p_1, \dots, p_n$ coprimos dos a dos, todo sistema $x \equiv a_i \pmod{p_i}$ tiene solución única módulo $M = p_1 \cdots p_n$.

Dadas dos congruencias, $X \equiv r_1 \pmod{m_1}$, $X \equiv r_2 \pmod{m_2}$

Sea $d = \text{mcd}(m_1, m_2)$

- El sistema tiene solución si y solo si $d|(r_2 - r_1)$
- Por Bezout, existen $x, y \in \mathbb{Z}$ que satisfacen $m_1x + m_2y = d$.

Y sabemos que $X_0 = r_1 + m_1x$

*Propiedad: $mcm(m_1, m_2) = \frac{m_1 m_2}{mcd(m_1, m_2)} = \frac{m_1 m_2}{d}$

*Por claridad entre nombres, en el código lo llamamos lcm (menor común múltiplo en inglés)

- Por Bezout, si x, y satisfacen $m_1x + m_2y = d$, una solución particular es:

$$x \equiv r_1 + m_1x \frac{r_2 - r_1}{d} \pmod{mcm(m_1, m_2)}$$

Esta fórmula se usa para el apartado opcional ya que implementa la combinación de dos congruencias en el caso no necesariamente coprimo (bibliografía)

Ahora, en un sistema de varias congruencias de la forma $a_i x \equiv b_i \pmod{p_i}$, donde los modulos no son coprimos, podemos simplificar cada congruencia a la forma $x \equiv r_i \pmod{m_i}$ con la explicación del apartado 'Ecuaciones lineales de congruencia'

Una vez conseguidas congruencias de la forma

$$x \equiv r_1 \pmod{m_1}, \quad x \equiv r_2 \pmod{m_2}, \quad \dots, \quad x \equiv r_n \pmod{m_n}$$

Podemos empezar a 'fusionar' las congruencias iterativamente de dos en dos usando la fórmula de arriba, donde se va acumulando la solución y su módulo sin tener que hacerlo todo de golpe.

Aplicación de la teoría a las funciones...

3.11.1. Resolver_lineal

$$\text{resolver_sistema_congruencias}(\text{alist}, \text{blist}, \text{plist}) \rightarrow (r, m)$$

1. Validamos que los tamaños de las listas cuadren
2. Simplifica la primera ecuación con *resolver_lineal*
3. Simplifica el resto de ecuaciones también con *resolver_lineal* y fusiona las congruencias con *tcr_dos*
4. Devuelve $\text{mod}(r, m), m$ que resuelve el sistema $a_i x \equiv b_i \pmod{p_i}$

Optimizaciones

- Uso *bezout* en vez de calcular las inversas (nuestro Euclides es óptimo)
- Trabajar con $m_1 = \frac{m}{d}$ en vez de con m simplifica el modulo a más pequeño que hace que el resto de 'fusiones' con *tcr* sean más rápidas
- Como usamos el módulo $mcm(m_1, m_2)$ en vez de (m_1, m_2) , el módulo no crece tan rápido cuando estamos acumulando y luego hay que simplificar menos

3.11.2. Tcr_dos

$tcrdos(r_1, m_1, r_2, m_2) \rightarrow (res, lcm)$

1. Comprueba que los dos módulos sean > 0
2. Calcula bezout de (m_1, m_2) con $m_1x + m_2y = d = mcd(m_1, m_2)$
3. Chequea que sea compatibles, es decir $d|(r_2 - r_1)$
4. Encontramos que $M = mcm(m_1, m_2)$
5. Calcula la solución con
6. $x \equiv r_1 + m_1x \frac{r_2 - r_1}{d} \pmod{mcm(m_1, m_2)}$

3.11.3. Mod

$Mod(a, m) \rightarrow \text{int}$

Devuelve el resto de a módulo m . Además, es siempre un número en $[0, m - 1]$ ya que muchos números son equivalentes al hacerles $mod\ m$ porque dejan el mismo resto después de simplificar. Muy simple solo para quede más limpia la función principal.

3.12. Carmichael

La conjetura de Carmichael $\lambda(n)$ de un entero positivo n es el entero positivo más pequeño m tal que $a^m \equiv 1 \pmod{n}$

Se cumple para cada entero coprimo con n .

$$\lambda(n) = \begin{cases} p^{e-1}(p-1) & si\ e = p^e, p \leq 3 \text{ o } e \leq 2 \\ 2^{e-2} & si\ e = 2^e \leq 3 \\ mcm(\lambda(p_1^{e_1}), \lambda(p_2^{e_2}), \dots, \lambda(p_t^{e_t})) & si\ n = \prod_{i=1}^t p_i^{e_i} \end{cases}$$

A primera vista, podemos ver que la función de Carmichael y la función de Euler se parecen.

Investigando un poco más la relación entre ambas llegamos a que:

- Siempre $\lambda(n) | \varphi(n) \therefore \lambda(n) \leq \varphi(n)$
- Coinciden para muchos casos: si n es primo p , o potencia prima impar p^e , entonces

$$\lambda(p^e) = \varphi(p^e) = (p-1)p^{e-1}$$

- Difieren en potencias de 2 grandes y en compuestos generales

- Para un n general:

$$\lambda(n) = \text{mcm}(\lambda(p_1^{e_1}), \dots, \lambda(p_t^{e_t})), \quad \varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

Y esto tiene consecuencias:

- Teorema de Euler: para todo a coprimo con n , $a^{\varphi(n)} \equiv 1 \pmod{n}$
- Carmichael (más fuerte para "todos a "): $a^{\lambda(n)} \equiv 1 \pmod{n}$ para todo a coprimo con n , y $\lambda(n)$ es el menor exponente común que lo garantiza

Adaptación de la función de Carmichael en *carmichael*:

1. Validaciones y casos triviales:
 - λ no está definida para $n \leq 0$
 - $\lambda(1) = 1$ por convenio
2. Factorizamos n y lo guardamos en el diccionario de su descomposición ya que necesitamos los pares (p, e) de n para poder aplicarle la fórmula de λ para los $n = 2$
3. Implementación de la fórmula matemática de Carmichael para potencias primas:

$$\lambda(p^e) = \begin{cases} 1 & (p = 2, e = 1), \\ 2 & (p = 2, e = 2), \\ 2^{e-2} & (p = 2, e \geq 3), \\ (p-1)p^{e-1} & (p \text{ impar}). \end{cases}$$

4. Actualizo $\lambda(lam)$ usando el menor común múltiplo de lam y $\lambda(p^e)$ (lam_pe) sabiendo que $\text{mcm}(a, b) = \frac{ab}{\text{mcd}(a, b)}$

Optimizaciones

- Al reducir n a potencias primas, puedo aplicar la fórmula cerrada para primos
- Como nuestra función *factorizar* es óptima, solo recorro los primos distintos que aparecen hasta n , no todos.

4. BIBLIOGRAFÍA

HackerRank. (s. f.). *Solve Regex*. Recuperado de <https://www.hackerrank.com/domains/regex>

AcademiaLab. (s. f.). *Exponenciación modular*. Recuperado de <https://academia-lab.com/enciclopedia/exponenciacion-modular/>

Wikipedia. (s. f.). *Exponenciación binaria*. En *Wikipedia, la enciclopedia libre*. Recuperado de https://es.wikipedia.org/wiki/Exponenciaci%C3%B3n_binaria

Wikipedia. (s. f.). *Inverso multiplicativo (aritmética modular)*. En *Wikipedia, la enciclopedia libre*. Recuperado de [https://es.wikipedia.org/wiki/Inverso_multiplicativo_\(aritm%C3%A9tica_modular\)](https://es.wikipedia.org/wiki/Inverso_multiplicativo_(aritm%C3%A9tica_modular))

AcademiaLab. (s. f.). *Criterio de Euler*. Recuperado de <https://academia-lab.com/enciclopedia/criterio-de-euler/>

Ramírez, K. (s. f.). *Anillos* [Presentación en PDF]. Recuperado de <https://www.kramirez.net/Discretas/Material/Presentaciones/anillos.pdf>

Euclides. (2009-10). *Apuntes: álgebra básica, tema 8* [PDF]. Recuperado de <http://euclides.us.es/da/apuntes/abasica/t8-2009-10.pdf>

Pfafner. (2021). *Aula 15* [PDF]. Recuperado de <https://pfafner.github.io/cr2021/aulas/Aula15.pdf>

Raji. (s. f.). *Símbolo de Legendre*. En *Teoría de números elementales* (Sección 5.05). Recuperado de [https://espanol.libretexts.org/Bookshelves/Matematicas/Combinatoria_y_Matematicas_Discretas/Teor%C3%ADa_de_n%C3%BAmeros_elementales_\(Raji\)/05%3A_Ra%C3%ADces_Primitivas_y_Residuos_Cuadr%C3%A1ticos/5.05%3A_S%C3%ADmbolo_Legendre](https://espanol.libretexts.org/Bookshelves/Matematicas/Combinatoria_y_Matematicas_Discretas/Teor%C3%ADa_de_n%C3%BAmeros_elementales_(Raji)/05%3A_Ra%C3%ADces_Primitivas_y_Residuos_Cuadr%C3%A1ticos/5.05%3A_S%C3%ADmbolo_Legendre)

Ecured. (s. f.). *Símbolo de Legendre (teoría de los números)*. Recuperado de [https://www.ecured.cu/S%C3%ADmbolo_de_Legendre_\(teor%C3%ADa_de_los_n%C3%BAmeros\)](https://www.ecured.cu/S%C3%ADmbolo_de_Legendre_(teor%C3%ADa_de_los_n%C3%BAmeros))

Wikipedia. (s. f.). *Símbolo de Legendre*. En *Wikipedia, la enciclopedia libre*. Recuperado de https://es.wikipedia.org/wiki/S%C3%ADmbolo_de_Legendre

Matetam. (s. f.). *Identidad de Bézout*. Recuperado de <https://www.matetam.com/glosario/teorema/identidad-bezout#:~:text=La%20demostraci%C3%B3n%20clásica%20inicia%20considerando%20el%20conjunto%20de,que%20ese%20elemento%20m%C3%ADnimo%20es%20el%20MCD%20%28a%2Cb%29%24.>

Frogames Formación. (s. f.). *Algoritmo de Euclides*. Recuperado de <https://cursos.frogamesformacion.com/pages/blog/algoritmo-de->

[euclides#:~:text=El%20Algoritmo%20de%20Euclides%20se%20basa%20en%20un,eficiente%2C%20si%20calculamos%20el%20residuo%20de%20su%20divisi%C3%B3n.](#)

Wikipedia. (s. f.). *Aritmética modular*. En *Wikipedia, la enciclopedia libre*. Recuperado de https://es.wikipedia.org/wiki/Aritm%C3%A9tica_modular

Wikipedia contributors. (2021, 16 de febrero). *Función de Carmichael*. En *Wikipedia, la enciclopedia libre*. Recuperado el 29 de septiembre de 2025, de

https://es.wikipedia.org/wiki/Funci%C3%B3n_de_Carmichael

forthright48. (2017, 18 de noviembre). *Chinese Remainder Theorem Part 2 – Non Coprime Moduli*. Forthright48. <https://forthright48.com/chinese-remainder-theorem-part-2-non-coprime-moduli/>